You can also write ordinary C# code in Razor templates by using this construct:

```
@{ /* C# code here */ }
```

Any code between the curly braces will be executed but won't be written to the response. In the listing, you're setting the title of the page by writing a key to the `ViewData` dictionary, but you aren't writing anything to the response at this point:

```
@{
    ViewData["Title"] = "Privacy Policy";
}
```

Another feature shown in this template is that you can dynamically write C# variables and expressions to the HTML stream using the `@` symbol. This ability to combine dynamic and static markup is what gives Razor Pages their power. In the example, you're fetching the `"Title"` value from the `ViewData` dictionary and writing the value to the response inside an `<h1>` tag:

```
<h1>@ViewData["Title"]</h1>
```

At this point, you might be a little confused by the template in listing 13.3 when it's compared with the output shown in figure 13.3. The title and the static HTML content appear in both the listing and figure, but some parts of the final web page don't appear in the template. How can that be?

Razor Pages have the concept of layouts, which are base templates that define the common elements of your application, such as headers and footers. The HTML of the layout combines with the Razor Page template to produce the final HTML that's sent to the browser. Layouts prevent you